

Practical No. 1

Definition : Write a program to demonstrate conditional statements using if if-else and if-elif-else.

Aim/ Objective:

- To understand the concept of conditional statements in Python.
- To learn how to use the if statement for decision-making.
- To learn how to use the if-else statement for choosing between two conditions.
- To learn how to use the if-elif-else statement for handling multiple conditions.
- To develop basic decision-making logic using Python.

Source Code:

```
num = int(input("Enter a number: "))
```

```
# 1. if statement
```

```
if num > 0:
```

```
    print("Number is Positive")
```

```
# 2. if-else statement
```

```
if num % 2 == 0:
```

```
    print("Number is Even")
```

```
else:
```

```
    print("Number is Odd")
```

```
# 3. if-elif-else statement
```

```
if num > 0:
```

```
    print("Number is greater than Zero")
```

```
elif num < 0:
```

```
    print("Number is less than Zero")
```

```
else:
```

```
    print("Number is Zero")
```

Output:

Enter a number: 7

Number is Positive

Number is Odd

Number is greater than Zero

Conclusion:

The program successfully demonstrates the use of conditional statements in Python. It uses if, if-else, and if-elif-else to check different conditions and make decisions based on the given number. This helps in understanding the basic decision-making process in Python programming.

Practical No.2

Definition: Write a program to check whether a number is positive negative or zero using nested conditions.

Aim & Objective:

- To understand the concept of nested if-else statements in Python.
- To learn how to check multiple conditions using nested conditions.
- To determine whether a number is positive, negative, or zero.
- To develop basic decision-making logic in Python.

Source Code:

```
num = int(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Number is Zero")
    else:
        print("Number is Positive")
else:
    print("Number is Negative")
```

Output:

Enter a number: 45

Number is Positive

Conclusion:

The program successfully checks whether the given number is positive, negative, or zero using nested conditional statements. It helps in understanding how one if-else statement can be placed inside another if-else statement for making multiple decisions.

Practical No.3

Definition: Write a program to generate a multiplication table using a for loop.

Aim & Objective:

- To understand the concept of the for loop in Python.
- To learn how to repeat a set of statements multiple times.
- To generate a multiplication table using iteration.
- To develop basic looping and programming logic.

Source Code:

```
num = int(input("Enter number: "))
```

```
for i in range(1, 11):
```

```
    print(num * i)
```

Output:

Enter number: 7

7

14

21

28

35

42

49

56

63

70

Conclusion:

The program successfully generates the multiplication table of a given number using a for loop. It helps in understanding how loops are used to perform repetitive tasks efficiently in Python.

Practical No.4

Definition: Write a program to find the sum of digits of a number using a while loop.

Aim & Objective:

- To understand the concept of the while loop in Python.
- To learn how to extract individual digits from a number.
- To calculate the sum of all digits of a given number.
- To develop basic looping and problem-solving logic in Python.

Source Code:

```
num = int(input("Enter a number: "))
sum = 0
while num > 0:
    digit = num % 10
    sum = sum + digit
    num = num // 10
print("Sum of digits =", sum)
```

Output:

Enter a number: 4

Sum of digits = 4

Conclusion:

The program successfully calculates the sum of the digits of a given number using a while loop. It helps in understanding how loops can be used to process the digits of a number repeatedly until all digits are processed.

Practical No.5

Definition: Write a program to demonstrate the use of break continue and pass statements.

Aim & Objective:

- To understand the purpose of the break statement.
- To learn how the continue statement skips the current iteration.
- To understand the use of the pass statement as an empty statement.
- To understand how these statements control the flow of loops in Python.

Source Code:

```
for i in range(1, 11):
```

```
    if i == 5:
```

```
        break
```

```
    print(i)
```

Output:

1

2

3

4

Conclusion:

The program successfully demonstrates the use of break, continue, and pass statements in Python. These statements help in controlling the execution and flow of loops according to different requirements.

Practical No.6

Definition: Write a program to iterate over lists strings and dictionaries using loops.

Aim & Objective:

- To understand the concept of iteration in Python.
- To learn how to use loops with lists.
- To learn how to iterate through characters of a string.
- To learn how to iterate through keys and values of a dictionary.
- To develop basic looping and data-handling skills in Python.

Source Code:

List

```
numbers = [10, 20, 30, 40]
```

```
for i in numbers:
```

```
    print(i)
```

String

```
name = "ARJUN"
```

```
for i in name:
```

```
    print(i)
```

Dictionary

```
student = {"Name": "Arjun", "Age": 21}
```

```
for key, value in student.items():
```

```
    print(key, value)
```

Output:

10

20

30

40

A

R

J

U

N

Name Arjun

Age 21

Conclusion:

The program successfully demonstrates how to iterate over lists, strings, and dictionaries using loops. It helps in understanding how loops can be used to access and process each element of different Python data structures.

Practical No.7

Definition: Write a program to demonstrate list dictionary and set comprehensions.

Aim & Objective:

- To understand the concept of comprehensions in Python.
- To learn how to create a list using list comprehension.
- To learn how to create a dictionary using dictionary comprehension.
- To learn how to create a set using set comprehension.
- To write Python code in a simple and efficient way.

Source Code:

```
# List Comprehension
```

```
numbers = [1, 2, 3, 4, 5]
```

```
square = [x * x for x in numbers]
```

```
print("List:", square)
```

```
# Dictionary Comprehension
```

```
numbers = [1, 2, 3, 4, 5]
```

```
square_dict = {x: x * x for x in numbers}
```

```
print("Dictionary:", square_dict)
```

```
# Set Comprehension
```

```
numbers = [1, 2, 3, 4, 5]
```

```
square_set = {x * x for x in numbers}
```

```
print("Set:", square_set)
```

Output:

List: [1, 4, 9, 16, 25]

Dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Set: {1, 4, 9, 16, 25}

Conclusion:

The program successfully demonstrates list, dictionary, and set comprehensions in Python. Comprehensions provide a simple and efficient way to create and process data collections using fewer lines of code.

Practical No.8

Definition: Write a program to illustrate variable scope using local global and nonlocal variables.

Aim & Objective:

- To understand the concept of variable scope in Python.
- To learn about local variables inside a function.
- To learn about global variables accessible throughout the program.
- To understand the use of nonlocal variables in nested functions.
- To understand how Python manages variables in different scopes.

Source Code:

```
# Global variable
```

```
x = 10
```

```
def outer():
```

```
    # Nonlocal variable
```

```
    y = 20
```

```
    def inner():
```

```
        # Local variable
```

```
z = 30
```

```
print("Global:", x)
print("Nonlocal:", y)
print("Local:", z)
```

```
inner()
```

```
outer()
```

Output:

Global: 10

Nonlocal: 20

Local: 30

Conclusion:

The program successfully demonstrates local, global, and nonlocal variables in Python. It helps in understanding how variables are accessed and modified in different parts of a program.

Practical No.9

Definition: Write a program to demonstrate iterators and iterables in Python.

Aim & Objective:

- To understand the concept of iterables and iterators.
- To learn the use of `iter()` and `next()` functions.
- To access elements one by one using an iterator.

Source Code:

```
# Iterable
numbers = [10, 20, 30]

# Iterator
it = iter(numbers)

print(next(it))
print(next(it))
print(next(it))
```

Output:

10

20

30

Conclusion:

Thus, we successfully demonstrated iterators and iterables in Python using `iter()` and `next()` functions.